

Another Mother — React/PWA Frontend Architecture Guide

Version 1.0 | MVP Phase 1

Philosophy: Mobile-First, One-Action-Per-Screen, Soft + Strong + Safe

1. ARCHITECTURE PRINCIPLES

1.1 Core UX Philosophy

Principle	Implementation	Anti-Pattern (Avoid)
One-action-per-screen	Every screen has ONE primary CTA	Forms with 12 fields on one page
Timeline-first	Home screen shows upcoming events chronologically	Dashboard with 8 widget grids
Action cards	Jobs, messages, alerts as swipeable cards	Tables, lists without clear actions
Bottom nav	3-5 persistent tabs at thumb-reach	Hamburger menu hiding everything
Field interruption	Save progress at every step, never lose work	“You must complete all fields”
Priority stacking	Most urgent/important items at top	Alphabetical or chronological only
Swipe actions	Swipe to accept, decline, archive, favourite	Long-press or hidden menus
Softer feel	Rounded corners, warm gradients, breathing room	Sharp edges, dense data, corporate blue

1.2 Technical Stack

Layer	Technology
Framework	React 19 + TypeScript
Build Tool	Vite 6 (fast HMR, optimal bundle)
State Management	Zustand (lightweight, no boilerplate)
Data Fetching	TanStack Query (React Query) v5
Routing	React Router v7 (data routers)
Styling	Tailwind CSS v4 + shadcn/ui components
Animation	Framer Motion (gestures, transitions)
Maps	Mapbox GL JS (privacy-friendly, cheaper than Google)
Payments	Stripe.js + Stripe Elements
PWA	Vite PWA Plugin (service worker, manifest)
Real-time	Socket.io Client (WebSocket events)
Forms	React Hook Form + Zod validation
Icons	Lucide React (consistent, lightweight)
Images	Cloudflare Images (optimised CDN)

1.3 Why This Stack

Choice	Why	Alternative Rejected
Zustand	1KB, no providers/wrappers, hooks-based	Redux (boilerplate), Context (performance)
TanStack Query	Caching, background sync, optimistic updates	SWR (less feature-rich), Apollo (GraphQL overkill)
Tailwind + shadcn	Utility-first speed + accessible primitives	Material UI (too corporate), styled-components (runtime cost)
Framer Motion	Declarative gestures, layout animations, exit transitions	CSS transitions (limited), GSAP (heavy)
Mapbox	\$0.50/1000 loads vs Google \$7/1000, better privacy	Google Maps (expensive, tracking)

2. PROJECT STRUCTURE

```
another-mother-web/
├─ public/
│  ├─ manifest.json          # PWA manifest
│  ├─ icons/
│  │  ├─ icon-192x192.png
│  │  ├─ icon-512x512.png
│  │  └─ apple-touch-icon.png
│  └─ sw.js                  # Service worker (auto-generated by Vite PWA)
├─ src/
│  ├─ main.tsx               # Entry point, providers wrap
│  ├─ App.tsx                # Root router, auth gate, layout shell
│  ├─ index.css              # Tailwind directives, custom properties
│  │
│  ├─ api/                   # API layer (TanStack Query + fetch)
│  │  ├─ client.ts           # Axios/fetch instance with auth interceptors
│  │  └─ queries/            # Query definitions
```

```

| | | | └─ useJobs.ts
| | | | └─ useProviders.ts
| | | | └─ useMessages.ts
| | | | └─ useEarnings.ts
| | | └─ mutations/ # Mutation definitions
| | | | └─ useCreateJob.ts
| | | | └─ useAcceptOffer.ts
| | | | └─ useCompleteJob.ts
| | | | └─ useSendMessage.ts
| | └─ keys.ts # Query key factory (centralised cache keys)
|
| └─ components/ # Shared UI primitives
| | └─ layout/
| | | └─ BottomNav.tsx # Persistent bottom navigation
| | | └─ TopBar.tsx # Context-aware header (title + actions)
| | | └─ ScreenShell.tsx # Safe area + scroll container + keyboard handling
| | | └─ TabLayout.tsx # Swipeable tab container
| | └─ cards/
| | | └─ JobCard.tsx # Timeline card (swipeable)
| | | └─ ProviderCard.tsx # Discovery card (tap to expand)
| | | └─ MessageCard.tsx # Chat bubble
| | | └─ OfferCard.tsx # Provider offer (accept/decline swipe)
| | | └─ EarningCard.tsx # Earnings summary
| | └─ inputs/
| | | └─ TextInput.tsx # Floating label, validation, help text
| | | └─ DatePicker.tsx # Native date + time picker wrapper
| | | └─ AddressInput.tsx # Google Places/Mapbox geocoding
| | | └─ FileUpload.tsx # Document upload with preview
| | | └─ ServiceSelector.tsx # Horizontal scrollable category pills
| | └─ feedback/
| | | └─ Toast.tsx # Soft toast (no aggressive popups)
| | | └─ LoadingState.tsx # Skeleton screens (never spinners on white)
| | | └─ EmptyState.tsx # Illustration + gentle CTA
| | | └─ ErrorState.tsx # Retry button + support contact
| | └─ actions/
| | | └─ PrimaryButton.tsx # Full-width, rounded, gradient
| | | └─ SecondaryButton.tsx # Outlined, less prominent
| | | └─ SwipeAction.tsx # Swipe-to-reveal action buttons
| | | └─ FloatingAction.tsx # FAB for primary creation action
|
| └─ screens/ # One-action-per-screen pages
| | └─ family/
| | | └─ HomeScreen.tsx # Timeline: upcoming jobs, quick actions
| | | └─ PostJobScreen.tsx # Step-by-step job creation wizard
| | | └─ JobDetailScreen.tsx # Job status, provider, messages
| | | └─ ProviderProfileScreen.tsx # Public provider view
| | | └─ SavedProvidersScreen.tsx # "My Mothers" list
| | | └─ MessagesScreen.tsx # Conversation list
| | | └─ ChatScreen.tsx # Single job chat
| | | └─ ReviewScreen.tsx # Post-job review
| | | └─ FamilyProfileScreen.tsx # Settings, children, payments

```

```

| | | └─ provider/
| | | |   └─ ProviderHomeScreen.tsx      # Dashboard: today's jobs, earnings
| | | |   └─ JobFeedScreen.tsx          # Available jobs (auto-assign + browse)
| | | |   └─ OfferScreen.tsx            # Full-screen offer (accept/decline)
| | | |   └─ MyJobsScreen.tsx           # Confirmed jobs list
| | | |   └─ JobActiveScreen.tsx        # En route → complete workflow
| | | |   └─ EarningsScreen.tsx         # Balance, history, payout
| | | |   └─ AvailabilityScreen.tsx     # Calendar block/unblock
| | | |   └─ VerificationScreen.tsx     # Document upload status
| | | |   └─ ProviderProfileScreen.tsx  # Bio, services, settings
| | | └─ shared/
| | | |   └─ OnboardingScreen.tsx       # Welcome, role selection
| | | |   └─ LoginScreen.tsx            # Email/password
| | | |   └─ RegisterScreen.tsx         # Account creation
| | | |   └─ NotFoundScreen.tsx        # Soft 404
| | | └─ admin/
| | | |   └─ AdminDashboardScreen.tsx
| | | |   └─ VerificationQueueScreen.tsx
| | | |   └─ JobOversightScreen.tsx
| | | |   └─ FinancialsScreen.tsx
| | | |   └─ IncidentScreen.tsx
| |
| └─ hooks/                                # Custom React hooks
| |   └─ useAuth.ts                        # Auth state, login/logout
| |   └─ useCurrentUser.ts                # Family or provider profile
| |   └─ useJobStatus.ts                  # Real-time job status via WebSocket
| |   └─ useLocation.ts                   # Geolocation with permission handling
| |   └─ useNotifications.ts              # Push notification permission + handling
| |   └─ useSwipeGesture.ts               # Swipe action detection
| |   └─ useKeyboard.ts                   # Keyboard show/hide detection
| |   └─ useOffline.ts                    # Network status, queue offline actions
| |
| └─ stores/                                # Zustand state stores
| |   └─ authStore.ts                     # User, tokens, onboarding status
| |   └─ uiStore.ts                       # Theme, bottom nav visibility, toasts
| |   └─ jobDraftStore.ts                  # Persisted job creation progress
| |   └─ notificationStore.ts              # Unread counts, push tokens
| |
| └─ types/                                # TypeScript definitions
| |   └─ api.ts                           # API request/response shapes
| |   └─ models.ts                         # Domain models (Job, Provider, etc.)
| |   └─ enums.ts                          # Status enums, service categories
| |   └─ stripe.ts                         # Stripe-specific types
| |
| └─ utils/                                # Pure functions
| |   └─ date.ts                           # Timezone-aware formatting, relative time
| |   └─ currency.ts                       # AUD formatting, fee calculations
| |   └─ validation.ts                     # Zod schemas for all forms
| |   └─ geo.ts                            # Distance calculation, suburb validation
| |   └─ constants.ts                      # App constants (fees, timeouts, limits)

```

```

|   └─ lib/                                # Third-party initialisation
|       └─ stripe.ts                       # Stripe.js initialisation
|       └─ socket.ts                      # Socket.io client setup
|       └─ mapbox.ts                      # Mapbox token + style config
|       └─ push.ts                       # Push notification service worker
|
|─ .env                                    # Environment variables
|─ vite.config.ts                         # Vite + PWA plugin config
|─ tailwind.config.ts                    # Theme tokens (colours, spacing, radii)
|─ tsconfig.json
|─ package.json

```

3. ROUTE STRUCTURE

3.1 Route Definitions

```

// src/App.tsx — Route configuration
import { createBrowserRouter, RouterProvider, Navigate } from 'react-router-dom';

const router = createBrowserRouter([
  // — PUBLIC ROUTES (no auth required) —
  {
    path: '/',
    element: <LandingPage />,           // Marketing site (or redirect to app)
  },
  {
    path: '/welcome',
    element: <OnboardingScreen />,      // Role selection: Family or Provider?
  },
  {
    path: '/login',
    element: <LoginScreen />,
  },
  {
    path: '/register',
    element: <RegisterScreen />,
    children: [
      { path: 'family', element: <FamilyRegistration /> },
      { path: 'provider', element: <ProviderRegistration /> },
    ],
  },
  {
    path: '/reset-password',
    element: <PasswordResetScreen />,
  },
  // — FAMILY ROUTES (auth + family role) —
  {
    path: '/family',
    element: <FamilyLayout />,          // BottomNav + TopBar + ScreenShell
  },

```

```

children: [
  { index: true, element: <Navigate to="/family/home" replace /> },
  { path: 'home', element: <HomeScreen /> },
  { path: 'post-job', element: <PostJobScreen /> },
  { path: 'jobs/:jobId', element: <JobDetailScreen /> },
  { path: 'jobs/:jobId/review', element: <ReviewScreen /> },
  { path: 'providers', element: <SavedProvidersScreen /> },
  { path: 'providers/:providerId', element: <ProviderProfileScreen /> },
  { path: 'messages', element: <MessagesScreen /> },
  { path: 'messages/:jobId', element: <ChatScreen /> },
  { path: 'profile', element: <FamilyProfileScreen /> },
],
},

// — PROVIDER ROUTES (auth + provider role) —
{
  path: '/provider',
  element: <ProviderLayout />,          // BottomNav + TopBar + ScreenShell
  children: [
    { index: true, element: <Navigate to="/provider/home" replace /> },
    { path: 'home', element: <ProviderHomeScreen /> },
    { path: 'jobs', element: <JobFeedScreen /> },
    { path: 'jobs/:jobId', element: <JobActiveScreen /> },
    { path: 'offers/:offerId', element: <OfferScreen /> },
    { path: 'earnings', element: <EarningsScreen /> },
    { path: 'availability', element: <AvailabilityScreen /> },
    { path: 'verification', element: <VerificationScreen /> },
    { path: 'profile', element: <ProviderProfileScreen /> },
  ],
},

// — ADMIN ROUTES (auth + admin role) —
{
  path: '/admin',
  element: <AdminLayout />,          // Sidebar + main content
  children: [
    { index: true, element: <Navigate to="/admin/dashboard" replace /> },
    { path: 'dashboard', element: <AdminDashboardScreen /> },
    { path: 'verification', element: <VerificationQueueScreen /> },
    { path: 'jobs', element: <JobOversightScreen /> },
    { path: 'financials', element: <FinancialsScreen /> },
    { path: 'incidents', element: <IncidentScreen /> },
  ],
},

// — CATCH ALL —
{ path: '*', element: <NotFoundScreen /> },
]);

```

3.2 Auth Gate & Role Guards

```
// src/components/layout/AuthGate.tsx
import { useAuthStore } from '@stores/authStore';
import { Navigate, useLocation } from 'react-router-dom';

export function AuthGate({ children, allowedRoles }: {
  children: React.ReactNode;
  allowedRoles?: ('family' | 'provider' | 'admin')[];
}) {
  const { user, isLoading } = useAuthStore();
  const location = useLocation();

  if (isLoading) return <LoadingState fullScreen />;

  if (!user) {
    return <Navigate to="/login" state={{ from: location.pathname }} replace />;
  }

  if (allowedRoles && !allowedRoles.includes(user.type)) {
    const redirectPath = user.type === 'family' ? '/family/home' : '/provider/home';
    return <Navigate to={redirectPath} replace />;
  }

  return <>{children}</>;
}

// Usage in layout components:
function FamilyLayout() {
  return (
    <AuthGate allowedRoles={['family']}>
      <ScreenShell>
        <TopBar />
        <Outlet />
        <BottomNav />
      </ScreenShell>
    </AuthGate>
  );
}
```

4. STATE MANAGEMENT (Zustand)

4.1 Auth Store

```
// src/stores/authStore.ts
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

interface User {
  id: string;
```

```

email: string;
type: 'family' | 'provider' | 'admin';
firstName: string;
onboardingCompleted: boolean;
}

interface AuthState {
  user: User | null;
  accessToken: string | null;
  refreshToken: string | null;
  isLoading: boolean;

  setAuth: (user: User, tokens: { access: string; refresh: string }) => void;
  updateTokens: (access: string) => void;
  logout: () => void;
  completeOnboarding: () => void;
}

export const useAuthStore = create<AuthState>()(
  persist(
    (set, get) => ({
      user: null,
      accessToken: null,
      refreshToken: null,
      isLoading: true,

      setAuth: (user, tokens) => set({
        user,
        accessToken: tokens.access,
        refreshToken: tokens.refresh,
        isLoading: false,
      }),

      updateTokens: (access) => set({ accessToken: access }),

      logout: () => {
        fetch('/api/v1/auth/logout', {
          method: 'DELETE',
          headers: { Authorization: `Bearer ${get().accessToken}` },
        });
        set({ user: null, accessToken: null, refreshToken: null, isLoading: false });
      },

      completeOnboarding: () => set((state) => ({
        user: state.user ? { ...state.user, onboardingCompleted: true } : null,
      })),
    }),
    {
      name: 'another-mother-auth',
      partialize: (state) => ({
        user: state.user,

```



```

        refreshToken: state.refreshToken
      }},
    }
  )
};

```

4.2 UI Store

```

// src/stores/uiStore.ts
import { create } from 'zustand';

interface Toast {
  id: string;
  message: string;
  type: 'success' | 'error' | 'info';
  duration?: number;
}

interface UIState {
  bottomNavVisible: boolean;
  topBarTitle: string;
  topBarActions: { icon: string; onClick: () => void }[];
  toasts: Toast[];
  activeModal: string | null;
  modalData: unknown;

  showToast: (toast: Omit<Toast, 'id'>) => void;
  dismissToast: (id: string) => void;
  setTopBar: (title: string, actions?: UIState['topBarActions']) => void;
  showBottomNav: (visible: boolean) => void;
  openModal: (name: string, data?: unknown) => void;
  closeModal: () => void;
}

export const useUIStore = create<UIState>((set) => ({
  bottomNavVisible: true,
  topBarTitle: '',
  topBarActions: [],
  toasts: [],
  activeModal: null,
  modalData: null,

  showToast: (toast) => {
    const id = Math.random().toString(36).substring(7);
    set((state) => ({ toasts: [...state.toasts, { ...toast, id }] }));
    setTimeout(() => {
      set((state) => ({ toasts: state.toasts.filter((t) => t.id !== id) }));
    }, toast.duration || 4000);
  },

  dismissToast: (id) => set((state) => ({

```

```

    toasts: state.toasts.filter((t) => t.id !== id),
  })),

  setTopBar: (title, actions = []) => set({ topBarTitle: title, topBarActions: actions }),
  showBottomNav: (visible) => set({ bottomNavVisible: visible }),
  openModal: (name, data) => set({ activeModal: name, modalData: data }),
  closeModal: () => set({ activeModal: null, modalData: null }),
}));

```

4.3 Job Draft Store (Field Interruption Protection)

```

// src/stores/jobDraftStore.ts
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

interface JobDraft {
  step: number;
  serviceCategory?: string;
  description?: string;
  address?: {
    line1: string;
    suburb: string;
    postcode: string;
    lat: number;
    lon: number;
  };
  scheduledDate?: string;
  startTime?: string;
  endTime?: string;
  isRecurring?: boolean;
  recurringPattern?: string;
  childrenPresent?: Array<{ name: string; age: number; notes?: string }>;
  specialInstructions?: string;
  preferredProviderId?: string;
  bookingMode?: 'auto-assign' | 'choose-provider';
}

interface JobDraftState {
  draft: JobDraft;
  updateStep: (step: number, data: Partial<JobDraft>) => void;
  clearDraft: () => void;
}

export const useJobDraftStore = create<JobDraftState>()(
  persist(
    (set) => ({
      draft: { step: 1 },

      updateStep: (step, data) => set((state) => ({
        draft: { ...state.draft, step, ...data },
      })),
    }),
  ),
);

```

```

      clearDraft: () => set({ draft: { step: 1 } })),
    ),
    {
      name: 'another-mother-job-draft',
    }
  )
);

```

5. SCREEN-BY-SCREEN COMPONENT MAP

5.1 Family: Home Screen (Timeline-First)

```

// src/screens/family/HomeScreen.tsx
import { useEffect } from 'react';
import { useUIStore } from '@stores/uiStore';
import { useJobs } from '@api/queries/useJobs';
import { JobCard } from '@components/cards/JobCard';
import { EmptyState } from '@components/feedback/EmptyState';
import { FloatingAction } from '@components/actions/FloatingAction';
import { motion, AnimatePresence } from 'framer-motion';

export function HomeScreen() {
  const { setTopBar } = useUIStore();
  const { data: jobs, isLoading } = useJobs({ status: 'upcoming' });

  useEffect(() => {
    setTopBar('Good morning, Sarah', [
      { icon: 'bell', onClick: () => navigate('/notifications') },
    ]);
  }, [setTopBar]);

  if (isLoading) return <LoadingState />;

  const today = jobs?.filter((j) => isToday(j.scheduledDate));
  const thisWeek = jobs?.filter((j) => isThisWeek(j.scheduledDate) && !isToday(j.scheduledDate));
  const later = jobs?.filter((j) => !isThisWeek(j.scheduledDate));

  return (
    <div className="flex flex-col h-full pb-20">
      { /* Priority Stack: Today's jobs */ }
      {today && today.length > 0 && (
        <section className="px-4 pt-4">
          <h2 className="text-sm font-medium text-navy-500 mb-2">Today</h2>
          <AnimatePresence>
            {today.map((job) => (
              <motion.div
                key={job.id}
                initial={{ opacity: 0, y: 20 }}

```

```

        animate={{ opacity: 1, y: 0 }}
        exit={{ opacity: 0, x: -100 }}
        layout
      >
        <JobCard job={job} variant="urgent" />
      </motion.div>
    )}}
  </AnimatePresence>
</section>
)}

{/* This week */}
{thisWeek && thisWeek.length > 0 && (
  <section className="px-4 pt-4">
    <h2 className="text-sm font-medium text-navy-500 mb-2">This week</h2>
    {thisWeek.map((job) => (
      <JobCard key={job.id} job={job} variant="default" />
    ))}
  </section>
)}

{/* Later */}
{later && later.length > 0 && (
  <section className="px-4 pt-4">
    <h2 className="text-sm font-medium text-navy-500 mb-2">Coming up</h2>
    {later.map((job) => (
      <JobCard key={job.id} job={job} variant="muted" />
    ))}
  </section>
)}

{/* Empty state */}
{jobs?.length === 0 && (
  <EmptyState
    illustration="empty-calendar"
    title="No jobs yet"
    message="Post your first job — it takes 2 minutes"
    action={{ label: 'Get help', onClick: () => navigate('/family/post-job') }}
  />
)}

{/* Quick action FAB */}
<FloatingAction
  icon="plus"
  label="Post a job"
  onClick={() => navigate('/family/post-job')}
/>
</div>
);
}

```

5.2 Job Card Component (Swipe Actions)

```
// src/components/cards/JobCard.tsx
import { useRef } from 'react';
import { motion, useMotionValue, useTransform, PanInfo } from 'framer-motion';
import { useNavigate } from 'react-router-dom';

interface JobCardProps {
  job: Job;
  variant: 'urgent' | 'default' | 'muted';
}

export function JobCard({ job, variant }: JobCardProps) {
  const navigate = useNavigate();
  const x = useMotionValue(0);
  const opacity = useTransform(x, [-150, 0, 150], [0.5, 1, 0.5]);
  const background = useTransform(
    x,
    [-150, 0, 150],
    ['rgb(239, 68, 68)', 'rgb(255, 255, 255)', 'rgb(34, 197, 94)']
  );

  const handleDragEnd = (event: MouseEvent | TouchEvent | PointerEvent, info: PanInfo) => {
    if (info.offset.x > 100 && job.status === 'offered') {
      acceptJob(job.id);
    } else if (info.offset.x < -100) {
      cancelJob(job.id);
    } else {
      x.set(0);
    }
  };

  const cardStyles = {
    urgent: 'bg-gradient-to-r from-pink-50 to-white border-l-4 border-pink-400',
    default: 'bg-white border border-navy-100',
    muted: 'bg-navy-50/50 border border-navy-100',
  };

  return (
    <motion.div
      style={{ x, opacity, background }}
      drag="x"
      dragConstraints={{ left: 0, right: 0 }}
      dragElastic={0.2}
      onDragEnd={handleDragEnd}
      onClick={() => navigate(`/family/jobs/${job.id}`)}
      className={`relative rounded-2xl p-4 mb-3 shadow-sm ${cardStyles[variant]} active:scale-[0.98]
        transition-transform`}
    >
      <div className="flex items-center justify-between mb-2">
        <span className={`text-xs font-semibold px-2 py-1 rounded-full ${`
```

```

    job.status === 'confirmed' ? 'bg-green-100 text-green-700' :
    job.status === 'matching' ? 'bg-amber-100 text-amber-700' :
    'bg-navy-100 text-navy-600'
  `}>
    {formatStatus(job.status)}
  </span>
  <span className="text-xs text-navy-400">
    {formatRelativeTime(job.startTime)}
  </span>
</div>

<h3 className="font-semibold text-navy-900 mb-1">{job.title}</h3>

{job.provider && (
  <div className="flex items-center gap-2 mt-2">
    <img
      src={job.provider.photoUrl}
      alt={job.provider.displayName}
      className="w-8 h-8 rounded-full object-cover"
    />
    <div>
      <p className="text-sm font-medium text-navy-800">{job.provider.displayName}</p>
      <p className="text-xs text-navy-400">
        {job.provider.rating} • {job.provider.distanceKm}km away
      </p>
    </div>
  </div>
)}
</motion.div>
);
}

```

5.3 Post Job Screen (Step Wizard)

```

// src/screens/family/PostJobScreen.tsx
import { useState } from 'react';
import { useJobDraftStore } from '@/stores/jobDraftStore';
import { useUIStore } from '@/stores/uiStore';
import { useCreateJob } from '@/api/mutations/useCreateJob';
import { motion, AnimatePresence } from 'framer-motion';

const STEPS = [
  { id: 'service', title: 'What do you need help with?' },
  { id: 'when', title: 'When do you need it?' },
  { id: 'where', title: 'Where should they go?' },
  { id: 'details', title: 'Any special instructions?' },
  { id: 'confirm', title: 'Ready to book?' },
];

export function PostJobScreen() {
  const { draft, updateStep, clearDraft } = useJobDraftStore();

```

```

const { showToast } = useUIStore();
const createJob = useCreateJob();
const [currentStep, setCurrentStep] = useState(draft.step || 1);

const handleNext = (stepData: Partial<JobDraft>) => {
  updateStep(currentStep + 1, stepData);
  setCurrentStep((prev) => prev + 1);
};

const handleBack = () => {
  if (currentStep > 1) setCurrentStep((prev) => prev - 1);
};

const handleSubmit = async () => {
  try {
    await createJob.mutateAsync(draft);
    clearDraft();
    showToast({ message: 'Job posted! Finding your mum...', type: 'success' });
    navigate('/family/home');
  } catch (err) {
    showToast({ message: 'Something went wrong. Your progress is saved.', type: 'error' });
  }
};

return (
  <div className="flex flex-col h-full">
    { /* Progress indicator */ }
    <div className="px-4 pt-4 pb-2">
      <div className="flex gap-1">
        {STEPS.map((_, idx) => (
          <div
            key={idx}
            className={`h-1 flex-1 rounded-full transition-colors ${
              idx < currentStep ? 'bg-pink-400' : 'bg-navy-100'
            }`}
          />
        ))}
      </div>
      <p className="text-xs text-navy-400 mt-2">Step {currentStep} of {STEPS.length}</p>
    </div>

    { /* Step content */ }
    <div className="flex-1 overflow-y-auto px-4">
      <AnimatePresence mode="wait">
        <motion.div
          key={currentStep}
          initial={{ opacity: 0, x: 50 }}
          animate={{ opacity: 1, x: 0 }}
          exit={{ opacity: 0, x: -50 }}
          transition={{ duration: 0.2 }}
        >

```

```

    <h1 className="text-2xl font-bold text-navy-900 mb-6">
      {STEPS[currentStep - 1].title}
    </h1>

    {currentStep === 1 && <ServiceStep onNext={handleNext} />}
    {currentStep === 2 && <WhenStep onNext={handleNext} onBack={handleBack} />}
    {currentStep === 3 && <WhereStep onNext={handleNext} onBack={handleBack} />}
    {currentStep === 4 && <DetailsStep onNext={handleNext} onBack={handleBack} />}
    {currentStep === 5 && <ConfirmStep onSubmit={handleSubmit} onBack={handleBack} />}

    </motion.div>
  </AnimatePresence>
</div>
</div>
);
}

// — STEP 1: Service Selection —
function ServiceStep({ onNext }: { onNext: (data: Partial<JobDraft>) => void }) {
  const { data: categories } = useServiceCategories();

  return (
    <div className="space-y-3">
      {categories?.map((cat) => (
        <button
          key={cat.id}
          onClick={() => onNext({ serviceCategory: cat.slug })}
          className="w-full flex items-center gap-4 p-4 rounded-2xl bg-white border border-navy-100
            hover:border-pink-300 hover:bg-pink-50/30 transition-all active:scale-[0.98]"
        >
          <div className="w-12 h-12 rounded-xl bg-gradient-to-br from-pink-400 to-cyan-400
            flex items-center justify-center text-white">
            <Icon name={cat.icon} size={24} />
          </div>
          <div className="text-left">
            <h3 className="font-semibold text-navy-900">{cat.name}</h3>
            <p className="text-sm text-navy-400">{cat.description}</p>
          </div>
          <ChevronRight className="ml-auto text-navy-300" />
        </button>
      ))}
    </div>
  );
}

// — STEP 5: Confirm —
function ConfirmStep({ onSubmit, onBack }: { onSubmit: () => void; onBack: () => void }) {
  const { draft } = useJobDraftStore();
  const { data: estimate } = useJobEstimate(draft);

  return (
    <div className="space-y-6">

```



```

<div className="bg-white rounded-2xl p-5 border border-navy-100 space-y-4">
  <SummaryRow label="Service" value={draft.serviceCategory} />
  <SummaryRow label="When" value={` ${draft.scheduledDate} at ${draft.startTime}`} />
  <SummaryRow label="Where" value={draft.address?.suburb} />

  <div className="border-t border-navy-100 pt-4">
    <div className="flex justify-between text-sm">
      <span className="text-navy-500">Service fee</span>
      <span className="font-medium">${estimate?.serviceAmount}</span>
    </div>
    <div className="flex justify-between text-sm mt-1">
      <span className="text-navy-500">Platform fee (15%)</span>
      <span className="font-medium">${estimate?.platformFee}</span>
    </div>
    <div className="flex justify-between text-lg font-bold mt-3 pt-3 border-t border-navy-100">
      <span className="text-navy-900">Total hold</span>
      <span className="text-pink-500">${estimate?.totalAmount}</span>
    </div>
    <p className="text-xs text-navy-400 mt-2">
      You won't be charged until the job is complete.
      Cancel within 10 minutes for a full refund.
    </p>
  </div>

  <div className="flex items-center justify-center gap-4 text-xs text-navy-400">
    <span className="flex items-center gap-1">
      <Shield size={14} /> Secure payment
    </span>
    <span className="flex items-center gap-1">
      <Heart size={14} /> Verified mums
    </span>
  </div>

  <div className="flex gap-3 pt-4">
    <SecondaryButton onClick={onBack} className="flex-1">Back</SecondaryButton>
    <PrimaryButton onClick={onSubmit} className="flex-1 bg-gradient-to-r from-pink-500 to-cyan-500">
      Confirm & Post Job
    </PrimaryButton>
  </div>
</div>
);
}

```

5.4 Provider: Offer Screen (Full-Screen Decision)

```

// src/screens/provider/OfferScreen.tsx
import { useEffect, useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { useOffer } from '@api/queries/useOffer';
import { useAcceptOffer } from '@api/mutations/useAcceptOffer';

```

```

import { useDeclineOffer } from '@api/mutations/useDeclineOffer';
import { motion, AnimatePresence } from 'framer-motion';
import { useUIStore } from '@stores/uiStore';

export function OfferScreen() {
  const { offerId } = useParams();
  const navigate = useNavigate();
  const { showToast } = useUIStore();
  const { data: offer, isLoading } = useOffer(offerId!);
  const acceptOffer = useAcceptOffer();
  const declineOffer = useDeclineOffer();
  const [timeLeft, setTimeLeft] = useState(180);
  const [showDeclineReasons, setShowDeclineReasons] = useState(false);

  useEffect(() => {
    if (!offer || timeLeft <= 0) return;
    const timer = setInterval(() => setTimeLeft((prev) => prev - 1), 1000);
    return () => clearInterval(timer);
  }, [offer, timeLeft]);

  useEffect(() => {
    if (timeLeft === 0) handleDecline('expired');
  }, [timeLeft]);

  const handleAccept = async () => {
    try {
      await acceptOffer.mutateAsync(offerId!);
      showToast({ message: 'Job accepted!', type: 'success' });
      navigate(`/provider/jobs/${offer?.jobId}`);
    } catch {
      showToast({ message: "Couldn't accept — job may have been assigned elsewhere", type: 'error' });
    }
  };

  const handleDecline = async (reason: string) => {
    try {
      await declineOffer.mutateAsync({ offerId: offerId!, reason });
      showToast({ message: 'No worries — passed to next mum', type: 'info' });
      navigate('/provider/jobs');
    } catch {
      showToast({ message: 'Something went wrong', type: 'error' });
    }
  };

  if (isLoading) return <LoadingState />;
  if (!offer) return <EmptyState title="Offer expired" message="This job has been assigned to another
  ↪ mum" />;

  return (
    <div className="flex flex-col h-full bg-navy-900 text-white">
      { /* Countdown */ }
    </div>
  );
}

```

```

<div className="px-4 pt-4 flex items-center justify-between">
  <span className="text-sm text-navy-300">
    Auto-declines in {Math.floor(timeLeft / 60)}:{String(timeLeft % 60).padStart(2, '0')}
  </span>
  <div className="w-32 h-1 bg-navy-700 rounded-full overflow-hidden">
    <motion.div
      className="h-full bg-pink-400"
      initial={{ width: '100%' }}
      animate={{ width: `${(timeLeft / 180) * 100}%` }}
      transition={{ duration: 1, ease: 'linear' }}
    />
  </div>
</div>

{/* Job details */}
<div className="flex-1 px-4 pt-6 overflow-y-auto">
  <div className="mb-6">
    <span className="text-xs font-semibold uppercase tracking-wider text-cyan-400">
      {offer.serviceCategory}
    </span>
    <h1 className="text-2xl font-bold mt-1">{offer.title}</h1>
  </div>

  {/* Map thumbnail */}
  <div className="w-full h-40 rounded-2xl bg-navy-800 mb-6 overflow-hidden">
    <MapThumbnail lat={offer.location.lat} lon={offer.location.lon}
      ↪ distance={offer.location.distanceKm} />
  </div>

  {/* Family info */}
  <div className="bg-navy-800/50 rounded-2xl p-4 mb-4">
    <div className="flex items-center gap-3">
      <div className="w-10 h-10 rounded-full bg-pink-500/20 flex items-center justify-center">
        <Users size={20} className="text-pink-400" />
      </div>
      <div>
        <p className="font-medium">{offer.family.name}</p>
        <p className="text-sm text-navy-300">
          {offer.family.rating} • {offer.family.totalReviews} reviews
        </p>
      </div>
    </div>
  </div>

  {/* Earnings */}
  <div className="bg-gradient-to-r from-pink-500/20 to-cyan-500/20 rounded-2xl p-4 mb-4">
    <p className="text-sm text-navy-300">You'll earn</p>
    <p className="text-3xl font-bold text-white">${offer.earnings.amount}</p>
    <p className="text-sm text-navy-300 mt-1">Paid within 24hrs of completion</p>
  </div>

```

```

    { /* Distance */
    <div className="flex items-center gap-2 text-sm text-navy-300">
      <MapPin size={16} />
      <span>{offer.location.distanceKm}km from you • ~{offer.location.estimatedDriveMin} min
        ↪ drive</span>
    </div>
  </div>

  { /* Actions */
  <div className="px-4 pb-8 pt-4 space-y-3">
    <AnimatePresence>
      {!showDeclineReasons ? (
        <motion.div className="space-y-3" exit={{ opacity: 0, y: 20 }}>
          <PrimaryButton
            onClick={handleAccept}
            className="w-full h-14 text-lg bg-gradient-to-r from-pink-500 to-cyan-500"
          >
            Accept Job
          </PrimaryButton>
          <SecondaryButton
            onClick={() => setShowDeclineReasons(true)}
            className="w-full h-12 border-navy-600 text-navy-300 hover:text-white"
          >
            Decline
          </SecondaryButton>
        </motion.div>
      ) : (
        <motion.div
          initial={{ opacity: 0, y: 20 }}
          animate={{ opacity: 1, y: 0 }}
          className="space-y-2"
        >
          <p className="text-sm text-navy-300 mb-2">Why are you passing on this one?</p>
          {[ 'Too far away', 'Time conflict', 'Not in my comfort zone', 'Personal
            ↪ reason'].map((reason) => (
              <button
                key={reason}
                onClick={() => handleDecline(reason.toLowerCase().replace(/ /g, '_'))}
                className="w-full text-left p-3 rounded-xl bg-navy-800 text-sm text-navy-200
                  ↪ hover:bg-navy-700 transition-colors"
              >
                {reason}
              </button>
            ))}
          <button
            onClick={() => setShowDeclineReasons(false)}
            className="w-full text-center p-3 text-sm text-navy-400"
          >
            Go back
          </button>
        </motion.div>
      )
    }
  </div>

```

```

    ))
  </AnimatePresence>
</div>
</div>
);
}

```

6. BOTTOM NAVIGATION

```

// src/components/layout/BottomNav.tsx
import { useLocation, useNavigate } from 'react-router-dom';
import { motion } from 'framer-motion';
import { useUIStore } from '@stores/uiStore';

const FAMILY_TABS = [
  { path: '/family/home', label: 'Home', icon: 'home', badge: 'upcoming_jobs_count' },
  { path: '/family/post-job', label: 'Post', icon: 'plus-circle', isAction: true },
  { path: '/family/messages', label: 'Messages', icon: 'message-circle', badge: 'unread_messages' },
  { path: '/family/profile', label: 'Profile', icon: 'user' },
];

const PROVIDER_TABS = [
  { path: '/provider/home', label: 'Home', icon: 'home', badge: 'today_jobs_count' },
  { path: '/provider/jobs', label: 'Jobs', icon: 'briefcase', badge: 'pending_offers' },
  { path: '/provider/earnings', label: 'Earnings', icon: 'dollar-sign' },
  { path: '/provider/profile', label: 'Profile', icon: 'user' },
];

export function BottomNav() {
  const location = useLocation();
  const navigate = useNavigate();
  const { bottomNavVisible } = useUIStore();
  const { user } = useAuthStore();

  const tabs = user?.type === 'family' ? FAMILY_TABS : PROVIDER_TABS;
  const activeTab = tabs.find((t) => location.pathname.startsWith(t.path));

  if (!bottomNavVisible) return null;

  return (
    <motion.nav
      initial={{ y: 100 }}
      animate={{ y: 0 }}
      className="fixed bottom-0 left-0 right-0 bg-white/90 backdrop-blur-lg
        border-t border-navy-100 px-2 pb-safe z-50"
    >
      <div className="flex items-center justify-around h-16 max-w-lg mx-auto">
        {tabs.map((tab) => {
          const isActive = activeTab?.path === tab.path;

```

```

return (
  <button
    key={tab.path}
    onClick={() => navigate(tab.path)}
    className={`relative flex flex-col items-center justify-center w-16 h-full
      ${tab.isAction ? '-mt-4' : ''}`}
  >
    {tab.isAction ? (
      <div className="w-14 h-14 rounded-full bg-gradient-to-r from-pink-500 to-cyan-500
        flex items-center justify-center shadow-lg shadow-pink-500/30">
        <Icon name={tab.icon} size={24} className="text-white" />
      </div>
    ) : (
      <>
        <Icon
          name={tab.icon}
          size={24}
          className={isActive ? 'text-pink-500' : 'text-navy-400'}
        />
        <span className={`text-xs mt-1 ${isActive ? 'text-pink-500 font-medium' :
          'text-navy-400'}`}>
          {tab.label}
        </span>

        {isActive && (
          <motion.div
            layoutId="activeTab"
            className="absolute -bottom-0 w-8 h-1 bg-pink-500 rounded-full"
          />
        )}
      </>
    )}
  </button>

  {tab.badge && <TabBadge type={tab.badge} />}
</button>
);
}}}
</div>
</motion.nav>
);
}

```

7. PWA CONFIGURATION

7.1 Vite PWA Plugin

```

// vite.config.ts
import { defineConfig } from 'vite';

```

```

import react from '@vitejs/plugin-react';
import { VitePWA } from 'vite-plugin-pwa';

export default defineConfig({
  plugins: [
    react(),
    VitePWA({
      registerType: 'autoUpdate',
      includeAssets: ['favicon.ico', 'apple-touch-icon.png', 'masked-icon.svg'],
      manifest: {
        name: 'Another Mother',
        short_name: 'AnotherMother',
        description: 'The village rebuilt — trusted mum-to-mum support',
        theme_color: '#1a1a2e',
        background_color: '#ffffff',
        display: 'standalone',
        scope: '/',
        start_url: '/',
        orientation: 'portrait',
        icons: [
          {
            src: '/icon-192x192.png',
            sizes: '192x192',
            type: 'image/png',
            purpose: 'any maskable',
          },
          {
            src: '/icon-512x512.png',
            sizes: '512x512',
            type: 'image/png',
            purpose: 'any maskable',
          },
        ],
      },
    },
  ],
  workbox: {
    globPatterns: ['**/*.{js,css,html,ico,png,svg,woff2}'],
    runtimeCaching: [
      {
        urlPattern: /^https:\/\/api\.anothermother\.com\.au\/v1\/$/,
        handler: 'NetworkFirst',
        options: {
          cacheName: 'api-cache',
          expiration: { maxEntries: 100, maxAgeSeconds: 300 },
        },
      },
      {
        urlPattern: /^https:\/\/cdn\.anothermother\.com\.au\/$/,
        handler: 'CacheFirst',
        options: {
          cacheName: 'media-cache',
          expiration: { maxEntries: 200, maxAgeSeconds: 86400 * 30 },
        },
      },
    ],
  },
});

```

```

    },
    },
    ],
    },
    })),
    ],
  });

```

7.2 Service Worker Strategy

Resource	Strategy	Rationale
API responses	NetworkFirst	Always fresh, fallback to cache offline
Images/media	CacheFirst	Fast loading, update occasionally
App shell (JS/CSS)	StaleWhileRevalidate	Fast load, update in background
Fonts	CacheFirst	Rarely change

7.3 Offline Behaviour

```

// src/hooks/useOffline.ts
import { useEffect, useState } from 'react';
import { useUIStore } from '@stores/uiStore';

export function useOffline() {
  const [isOffline, setIsOffline] = useState(!navigator.onLine);
  const { showToast } = useUIStore();

  useEffect(() => {
    const handleOnline = () => {
      setIsOffline(false);
      showToast({ message: "You're back online!", type: 'success' });
    };

    const handleOffline = () => {
      setIsOffline(true);
      showToast({
        message: "You're offline. Some features may be limited.",
        type: 'info',
        duration: 6000
      });
    };

    window.addEventListener('online', handleOnline);
    window.addEventListener('offline', handleOffline);

    return () => {
      window.removeEventListener('online', handleOnline);
      window.removeEventListener('offline', handleOffline);
    };
  });
}

```



```
    }, [showToast]);

    return { isOffline };
}
```

8. DESIGN TOKENS (Tailwind Config)

```
// tailwind.config.ts
import type { Config } from 'tailwindcss';

const config: Config = {
  content: ['./src/**/*.ts', './src/**/*.tsx'],
  theme: {
    extend: {
      colors: {
        navy: {
          50: '#f5f5f8',
          100: '#e5e5ec',
          200: '#c9c9d9',
          300: '#a5a5b8',
          400: '#7a7a94',
          500: '#5a5a78',
          600: '#3d3d5c',
          700: '#2a2a45',
          800: '#1a1a2e',
          900: '#0f0f1a',
        },
        pink: {
          50: '#fdf2f8',
          100: '#fce7f3',
          200: '#fbc02d',
          300: '#f9a8d4',
          400: '#f472b6',
          500: '#ec4899',
          600: '#db2777',
        },
        cyan: {
          50: '#ecf9ff',
          100: '#cffffe',
          200: '#a5f3fc',
          300: '#67e8f9',
          400: '#22d3ee',
          500: '#00bcd4',
        },
        success: '#22c55e',
        warning: '#f59e0b',
        error: '#ef4444',
      },
      fontFamily: {
```

```

    sans: ['Inter', 'system-ui', 'sans-serif'],
    display: ['Playfair Display', 'serif'],
  },
  borderRadius: {
    '2xl': '1rem',
    '3xl': '1.5rem',
  },
  spacing: {
    'safe-top': 'env(safe-area-inset-top)',
    'safe-bottom': 'env(safe-area-inset-bottom)',
  },
  animation: {
    'slide-up': 'slideUp 0.3s ease-out',
    'fade-in': 'fadeIn 0.2s ease-out',
    'pulse-soft': 'pulseSoft 2s infinite',
  },
  keyframes: {
    slideUp: {
      '0%': { transform: 'translateY(20px)', opacity: '0' },
      '100%': { transform: 'translateY(0)', opacity: '1' },
    },
    fadeIn: {
      '0%': { opacity: '0' },
      '100%': { opacity: '1' },
    },
    pulseSoft: {
      '0%, 100%': { opacity: '1' },
      '50%': { opacity: '0.7' },
    },
  },
},
},
plugins: [],
};

export default config;

```

9. REAL-TIME INTEGRATION (WebSocket)

```

// src/lib/socket.ts
import { io, Socket } from 'socket.io-client';
import { useAuthStore } from '@stores/authStore';

let socket: Socket | null = null;

export function getSocket(): Socket {
  if (!socket) {
    const { accessToken } = useAuthStore.getState();

```

```

socket = io('wss://api.anothermother.com.au', {
  auth: { token: accessToken },
  transports: ['websocket'],
  reconnection: true,
  reconnectionAttempts: 5,
  reconnectionDelay: 1000,
});

socket.on('connect', () => console.log('WebSocket connected'));
socket.on('disconnect', (reason) => {
  console.log('WebSocket disconnected:', reason);
  if (reason === 'io server disconnect') socket?.connect();
});
}

return socket;
}

// Hook for real-time job status
export function useJobStatus(jobId: string) {
  const [status, setStatus] = useState<JobStatus | null>(null);
  const queryClient = useQueryClient();

  useEffect(() => {
    const socket = getSocket();
    socket.emit('subscribe', { channels: [`job:${jobId}`] });

    socket.on('job_status_changed', (data) => {
      if (data.jobId === jobId) {
        setStatus(data.newStatus);
        queryClient.invalidateQueries({ queryKey: ['jobs', jobId] });
      }
    });

    socket.on('provider_assigned', (data) => {
      if (data.jobId === jobId) {
        queryClient.setQueryData(['jobs', jobId], (old: Job) => ({
          ...old,
          provider: data.provider,
          status: 'confirmed',
        }));
      }
    });

    socket.on('new_message', (data) => {
      if (data.jobId === jobId) {
        queryClient.invalidateQueries({ queryKey: ['messages', jobId] });
      }
    });
  });

  return () => {

```

```
    socket.emit('unsubscribe', { channels: [`job:${jobId}`] });
    socket.off('job_status_changed');
    socket.off('provider_assigned');
    socket.off('new_message');
  };
}, [jobId, queryClient]);

return { status };
}
```

10. PERFORMANCE BUDGETS

Metric	Target	Max
First Contentful Paint (FCP)	< 1.0s	1.5s
Largest Contentful Paint (LCP)	< 2.0s	2.5s
Time to Interactive (TTI)	< 3.0s	4.0s
Cumulative Layout Shift (CLS)	< 0.05	0.1
Total Bundle Size (gzipped)	< 200KB	250KB
API Response Time (p95)	< 500ms	1s
WebSocket Latency	< 100ms	200ms

10.1 Bundle Optimisation

```
// vite.config.ts — Code splitting
export default defineConfig({
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          'react-vendor': ['react', 'react-dom', 'react-router-dom'],
          'stripe': ['@stripe/stripe-js'],
          'maps': ['mapbox-gl'],
          'animation': ['framer-motion'],
          'query': ['@tanstack/react-query'],
        },
      },
    },
  },
});
```

10.2 Image Optimisation

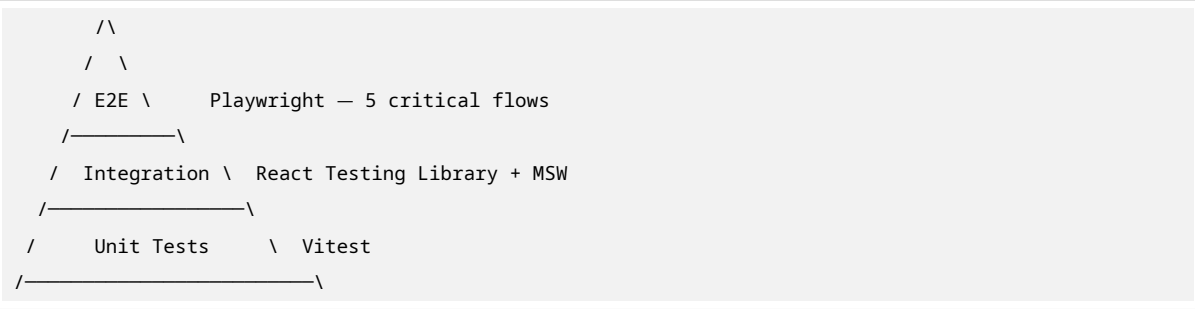
```
// Cloudflare Images URL builder
export function getOptimizedImageUrl(
  originalUrl: string,
  options: { width?: number; height?: number; quality?: number; format?: 'webp' | 'avif' }
): string {
```

```
const { width = 400, height, quality = 80, format = 'webp' } = options;
return `${originalUrl}?w=${width}&q=${quality}&f=${format}${height ? `&h=${height}` : ''}`;
}

// Usage
<img
  src={getOptimizedImageUrl(provider.photoUrl, { width: 200, height: 200 })}
  alt={provider.displayName}
  loading="lazy"
  className="w-12 h-12 rounded-full object-cover"
/>
```

11. TESTING STRATEGY

11.1 Testing Pyramid



11.2 Critical E2E Flows (Playwright)

Flow	Steps	Assertions
Family books auto-assign	Register → Post job → Payment → Provider assigned → Complete → Review	Job status transitions, payment hold, provider notification
Provider accepts offer	Login → Receive offer → Accept → Mark en route → Mark complete	Offer countdown, status updates, earnings update
Cancellation & refund	Book job → Cancel within 10 min → Verify refund	Full refund processed
Saved provider rebooking	Save provider → Book with preferred → Same provider assigned	Saved preference weighting
Offline resilience	Go offline → Attempt action → Come online → Action completes	Toast notification, queued action

11.3 Component Testing Example

```
// src/components/cards/__tests__/JobCard.test.tsx
```

```

import { render, screen, fireEvent } from '@testing-library/react';
import { JobCard } from '../JobCard';
import { BrowserRouter } from 'react-router-dom';

const mockJob = {
  id: 'job-123',
  title: 'School Pickup',
  status: 'confirmed',
  scheduledDate: '2026-05-20',
  startTime: '15:30',
  provider: {
    displayName: 'Sarah M.',
    photoUrl: 'https://cdn.example.com/sarah.jpg',
    rating: 4.9,
    distanceKm: 2.3,
  },
};

describe('JobCard', () => {
  it('renders job title and provider', () => {
    render(
      <BrowserRouter>
        <JobCard job={mockJob} variant="default" />
      </BrowserRouter>
    );

    expect(screen.getByText('School Pickup')).toBeInTheDocument();
    expect(screen.getByText('Sarah M.')).toBeInTheDocument();
    expect(screen.getByText('4.9$\\vcenter{{\\hbox{{\\includegraphics[height=\\ht\\strutbox]{{/usr/share/docmininer/emoji/2605.png}}}}}$' • 2.3km away')).toBeInTheDocument();
  });

  it('shows urgent styling for today jobs', () => {
    render(
      <BrowserRouter>
        <JobCard job={mockJob} variant="urgent" />
      </BrowserRouter>
    );
    const card = screen.getByRole('button');
    expect(card).toHaveClass('border-pink-400');
  });

  it('navigates to job detail on click', () => {
    const navigate = jest.fn();
    jest.spyOn(require('react-router-dom'), 'useNavigate').mockReturnValue(navigate);

    render(
      <BrowserRouter>
        <JobCard job={mockJob} variant="default" />
      </BrowserRouter>
    );
  });
});

```

```
fireEvent.click(screen.getByRole('button'));
expect(navigate).toHaveBeenCalledWith('/family/jobs/job-123');
});
});
```

12. ACCESSIBILITY CHECKLIST

Requirement	Implementation
Touch targets	All interactive elements 44×44px
Color contrast	All text meets WCAG AA (4.5:1 ratio)
Screen readers	Semantic HTML, ARIA labels on icons, aria-live for toasts
Focus management	Auto-focus on modal open, focus trap, restore on close
Reduced motion	prefers-reduced-motion disables animations
Font scaling	All sizes in rem, respects system font size
Keyboard navigation	Tab order logical, Enter/Space activate, Escape closes

```
// Accessibility hook for reduced motion
export function useReducedMotion() {
  const [reduced, setReduced] = useState(false);

  useEffect(() => {
    const media = window.matchMedia('(prefers-reduced-motion: reduce)');
    setReduced(media.matches);
    const handler = (e: MediaQueryListEvent) => setReduced(e.matches);
    media.addEventListener('change', handler);
    return () => media.removeEventListener('change', handler);
  }, []);

  return reduced;
}
```

13. POST-MVP FRONTEND FEATURES

Feature	Description	Complexity
In-app video calls	WebRTC for pre-job video chat	High
Advanced animations	Lottie for empty states, success	Medium
Dark mode	System-aware dark theme	Low
Voice input	Speech-to-text for job posting	Medium
Biometric auth	Face ID / fingerprint quick login	Medium
Widget (iOS/Android)	Home screen widget showing next job	High
Apple Watch	Quick glance at upcoming jobs	High
Share extensions	Share from other apps to create job	Medium

14. QUICK REFERENCE: COMPONENT DECISION TREE

```
Need a new screen?
├─ Is it family-facing?
│   ├─ Yes → src/screens/family/
│   └─ No → src/screens/provider/
├─ Does it need auth?
│   ├─ Yes → Wrap in AuthGate
│   └─ No → Add to public routes
├─ Is it a full-screen flow?
│   ├─ Yes → Use ScreenShell + BottomNav
│   └─ No → Use modal (sparingly)
├─ Does it collect data?
│   ├─ Yes → Use React Hook Form + Zod
│   └─ No → Static content
├─ Does it need real-time?
│   ├─ Yes → Use useJobStatus hook
│   └─ No → TanStack Query is enough
└─ Is it performance-critical?
    ├─ Yes → Lazy load + code split
    └─ No → Standard import
```

15. DEPLOYMENT CHECKLIST

Pre-Launch

- All environment variables set in VentraIP
- Stripe publishable key uses pk_live_
- API base URL points to production
- Sentry/Datadog error tracking configured
- Google Analytics / Meta Pixel installed
- PWA manifest validated
- Service worker caches correct assets
- HTTPS enforced (Cloudflare SSL)
- CSP headers configured
- robots.txt allows indexing
- sitemap.xml generated

Post-Launch Monitoring

- Core Web Vitals tracked (LCP < 2.5s, CLS < 0.1)
- Error rate < 0.1%
- API 500 rate < 0.01%
- PWA install rate tracked
- Push notification opt-in rate > 30%